DOCUMENTATION SECTION

ACTUS Insurance Extensions

How the ACTUS framework was extended to model insurance contracts — life insurance projection, Markov state transitions, and a domain-specific language for product rules.

- Life Insurance Projection on GPU
- Markov State Transition Model
- Domain-Specific Language for Product Rules
- LifeProjectionCube Data Structure

Table of Contents

- ACTUS Insurance Extensions
- Scenario split
  - Life Insurance Model
  - Syntax Reference
  - Markov State Transitions
  - Data References
  - DSL & Product Rules
  - Built-in Functions
  - Monte Carlo Simulation
  - Factor Definitions
  - Guard Rules
  - DSL Internals

ACTUS Insurance

ACTUS Insurance Extensions

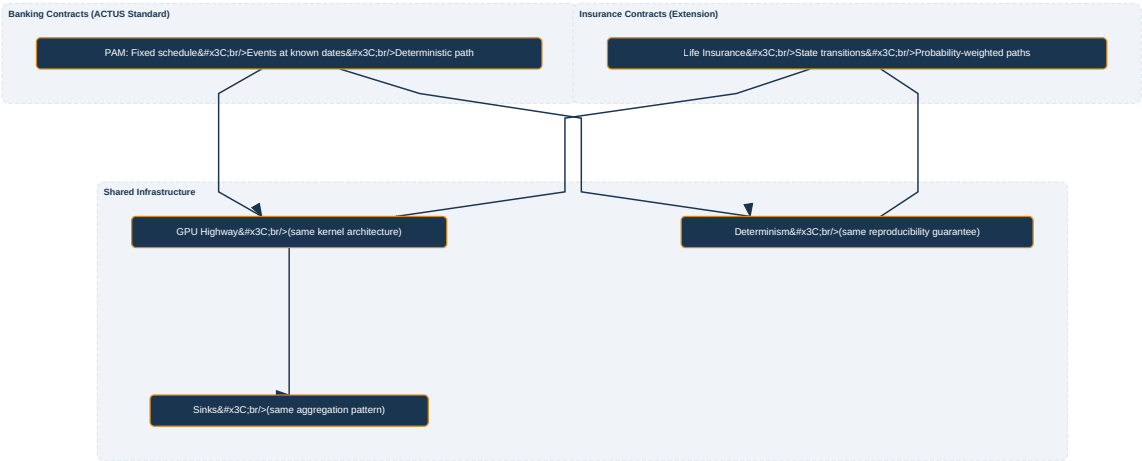
ACTUS Insurance Extensions

Why Insurance?

ACTUS was designed for banking contracts — loans, bonds, deposits — where the contract follows a fixed schedule. Insurance contracts are fundamentally different: a life insurance policy can be active, lapse, enter a grace period, trigger a death benefit, or be reinstated. The path a policy takes depends on probabilities, not a fixed schedule.

This project extends ACTUS to handle insurance contracts while preserving the standard's core property: **determinism**. The same policy, with the same actuarial tables and the same terms, always produces the same expected cash flows.

The Insurance Extension Architecture



The extension adds three new capabilities:

1. **Markov state transition model** — defines all possible states a policy can be in and the probabilities of moving between them
2. **Actuarial lookup tables** — mortality, lapse, and disability rates indexed by age, gender, duration, and other factors
3. **Domain-specific language (DSL)** — configurable product rules that allow actuaries to define new insurance products without changing the engine

How Insurance Differs from Banking

Aspect	Banking Contract (PAM)	Insurance Contract (Life)
Event schedule	Fixed: payments at known dates	Dynamic: transitions depend on probabilities
Path	One deterministic path	Multiple possible paths (active, lapsed, claimed, etc.)
State changes	Predictable (rate reset, maturity)	Probabilistic (death, lapse, disability)
Cash flows	Exact amounts	Expected values (probability-weighted averages)
Time step	Event-driven (irregular dates)	Regular intervals (monthly)
External data	Market rates	Actuarial tables (mortality, lapse, disability)

The key insight: while individual policy paths are uncertain, the **expected** cash flow — the probability-weighted average across all possible paths — is deterministic. Given the same actuarial assumptions, the expected cash flow is always the same.

Continue Reading

- Life Insurance Model — how life insurance policies are projected on the GPU
- Markov State Transitions — why the Markov model is used, how it steps forward, and how rules fill the gaps that tables cannot
- DSL & Product Rules — the domain-specific language for configurable insurance products
- Monte Carlo Simulation — running thousands of scenarios to build a distribution of outcomes and quantify tail risk

Scenario split

Scenario: Design Reasoning & Philosophy

Context: `ActusInsurance.Tests.GPU / ScenarioTests.cs`

Why Scenarios Exist

A large insurance or financial portfolio is never truly "known" — it is a projection forward in time from a fixed vantage point. Interest rates will move, prepayment speeds will shift, and credit spreads will widen or tighten in ways that no model can predict with certainty. The purpose of scenario testing is not to find the single correct answer but to understand the **distribution of possible futures** and, crucially, to understand how well yesterday's predictions match today's reality.

The Central Idea: A Moving Knowledge Boundary

Every projection is made from a **calculation date** (`calculationDate`). This date is the dividing line between what is known and what is uncertain:

- **Before the calculation date** — these events have already happened. The rate resets, coupon payments, and principal repayments that fall in this region are **facts**. They happened; the market has recorded them.
- **On or after the calculation date** — these events have not happened yet. They are **predictions**. The model applies scenario rates to estimate their value.

This boundary is enforced at the data level through the `ScenarioRateSeries` dual-value structure. Every time point in a scenario carries two independent rates:

```
(prior rate, after rate)  →  keyed by tick timestamp
```

When the system evaluates a rate-reset event (`RR`), it looks up whether that event falls before or after the calculation date and selects the appropriate value. Events in the past use the `prior` rate; events in the future use the `after` rate. The lookup itself uses the closest earlier entry as a fallback, so a single early-dated entry acts as a blanket override for the entire series.

What Makes Predictions Go Stale

Consider a projection made on **2022-01-01** for a five-year floating-rate portfolio. The model picks scenario rates for 2022–2027. Those rates feel reasonable at the time.

By **2023-01-01**, one year of reality has elapsed. The rates the model chose for 2022 are now facts — and they almost certainly do not match the scenarios exactly. If the model is never updated, its predictions for 2023–2027 are still anchored to the 2022 view of the world: they have **gone stale**.

Staleness has two dimensions:

1. **Temporal drift** — the longer time passes, the more "factual" history diverges from what the old scenarios assumed, eroding trust in the forward projections.
2. **Regime drift** — a structural shift (e.g. an abrupt rate-hiking cycle) can render the entire distributional shape of future scenarios implausible, not just their level.

Keeping the prior and after values separate is what makes it possible to detect and measure staleness. You can re-run the same portfolio with a new calculation date and new scenarios while leaving the historical portion unchanged, so differences in outcome are attributable purely to updated forward assumptions rather than to recomputed history.

Multiple Scenario Vintages: Measuring Adaptability

The most powerful insight comes not from running a single scenario set but from running **several sets anchored at different calculation dates** and comparing them.

Imagine three projection vintages:

Vintage	Calculation Date	Forward Assumption
V_1	2021-01-01	Rates stay low through 2025
V_2	2022-07-01	Rates rise 200 bps by 2023
V_3	2023-01-01	Rates plateau at elevated level

The `PayoffCube` captures this naturally. Its four-dimensional structure is indexed as:

```
cube[k, c, s, t]
  k = calc-date slice (vintage index)
```

```
    c = contract index
    s = scenario index within that vintage
    t = time-step on the shared output grid
```

When you align the slices along the time axis you can answer the question: **"At time t, how close was vintage V_n's prediction to the subsequent factual outcome?"**

If V_1 predicted that the portfolio would accrue interest at 2 % throughout 2022 but V_2 — already partially informed by reality — predicted 4 %, and the actual 2022 cash flows were at 4 %, then V_2 adapted well and V_1 failed to. This comparison is only possible because the prior/after boundary keeps historical facts frozen while allowing the forward assumptions to change between vintages.

In code this is expressed through `OutputPlan.RollingCalcDates`, which schedules the same portfolio to be evaluated once for each vintage:

```
var plan = OutputPlan.RollingCalcDates(
    new[] { calcDate2021, calcDate2022, calcDate2023 },
    timeGrid);
var cube = executor.Execute(contracts, rf, scenarioSet, plan);
```

Backtest Mode: Asserting the Past

When `after == prior` for every time point in a `ScenarioRateSeries`, the scenario contains **no forward uncertainty**. All events, past and future, use the same rate. This is **backtest mode**.

Backtest mode is useful in two situations:

1. **Regression testing** — fixing all rates to their known historical values should reproduce deterministic cash-flow schedules that match reference data exactly.
2. **Baseline anchoring** — before stressing the portfolio with hypothetical rate paths, you establish a "what actually happened" baseline. The difference between the backtest payoff and a stressed payoff is the pure scenario delta, with no noise from recomputing history.

The `BacktestModeTest` verifies the invariant: if every scenario in a set has `after == prior`, then all scenarios must produce identical payoffs regardless of how many there are, because there is no variation to exploit.

Monte Carlo Scenarios: Sampling the Forward Distribution

Rather than hand-crafting individual rate paths, `MonteCarloScenarioGenerator` samples a distribution of future rates by perturbing the base `after` values with log-normal noise. The `prior` values are left unchanged — consistent with the rule that the factual past is not touched by forward uncertainty.

Key design choices:

- **Seed determinism** — the same seed always produces the same set of paths. This means that adding a new scenario or changing a contract definition does not silently invalidate the reproducibility of existing runs. It also makes CI/CD comparisons stable.
- **Zero-volatility degeneracy** — with `volatility = 0`, every scenario collapses to the base after-rate path. This is the Monte Carlo equivalent of a deterministic forecast and acts as a natural sanity check.
- **Independent prior and after** — the generator perturbs only the `after` values. The `prior` series is kept at its base, ensuring that historical consistency is never broken by sampling variance.

The Portfolio Runner and Output Shape

The `ScenarioBatchExecutor` is the workhorse for a single calculation-date evaluation. It returns a flat array of `ScenarioOutcome` objects with shape `[numContracts × numScenarios]`, laid out so that all scenarios for contract `c` occupy a contiguous block starting at `c * numScenarios`. This layout makes it efficient to compute per-contract statistics (min, max, mean payoff across scenarios) using `ScenarioOutcomeAggregator.ContractPayoffStats`.

When multiple calculation dates are involved, `PayoffCubeExecutor` replaces the batch executor. Instead of a flat scalar per `(contract, scenario)` pair, it produces a full time-series of payoffs for each combination, forming the four-dimensional cube described above. Both executors share the same GPU kernel path through `PamV3Adapter` and `ScenarioBatchExecutor`, so numerical results are consistent across the two surfaces.

Summary of the Design

The scenario architecture is built around one governing insight: **the boundary between fact and prediction is not fixed in time — it advances as the world unfolds.** By encoding this boundary explicitly in every rate series, separating it from the contract terms,

and making it movable through the choice of calculation date, the system enables a kind of institutional memory: you can always re-run an old projection, compare it to the new one, and quantify exactly how much your assumptions have improved — or failed to improve — in the intervening period.

```
Past ──────────────┼────────────────── Future
        FACTS      │      PREDICTIONS
                   │
             calculationDate
              (movable)
```

This is not merely a testing concern. It is the foundation for any credible forward-looking risk analysis: know what you know, be honest about what you are guessing, and always be able to show your work as time moves on.

Life Insurance Model

Life Insurance Model

Overview

The life insurance projection model takes a portfolio of insurance policies and projects each one forward over a multi-year horizon (typically 30 years), computing expected cash flows at each time step. Unlike banking contracts where events happen at fixed dates, insurance projections step through time at regular monthly intervals, applying probabilistic state transitions at each step.

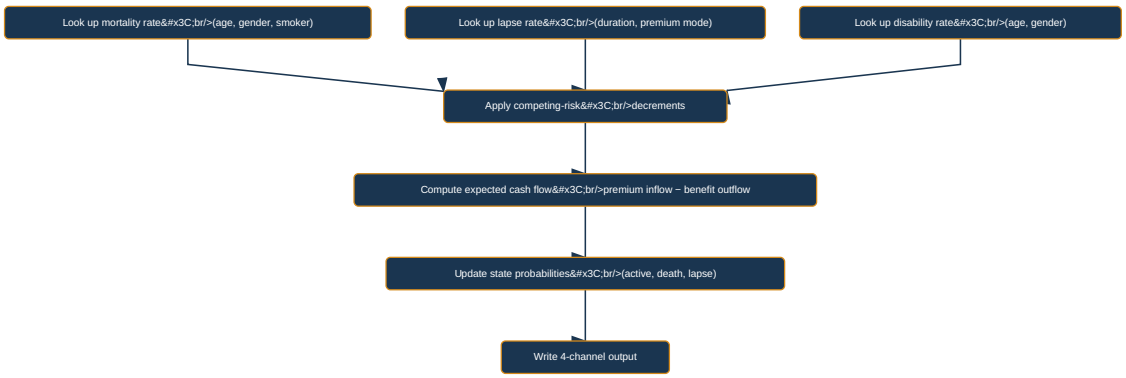
Contract Data

Each insurance policy is represented by a set of attributes:

Attribute	Description
ContractId	Unique policy identifier
CurrentState	Current policy state (Active, Lapsed, etc.)
SumAssured	Death benefit amount
PremiumAmount	Regular premium payment
PremiumMode	Payment frequency (Monthly, Quarterly, Annual)
AgeAtEvaluation	Insured person's age at the evaluation date
InsuredGender	Male, Female, or Unspecified
SmokerStatus	Whether the insured is a smoker
YearsInForce	How long the policy has been active
ExtraPremiumBps	Underwriting loading in basis points
ClaimsCount	Number of prior claims
LapseCount	Number of prior lapses

Projection Algorithm

At each monthly time step, the kernel performs these calculations for each policy:



Step 1: Look Up Hazard Rates

Three hazard rates are retrieved from pre-built lookup tables:

Mortality rate — based on the Gompertz-Makeham model: a parametric formula that increases exponentially with age. The base rate is adjusted for gender (female rates are 85% of male) and loaded for smoker status (1.35× for smokers) and underwriting adjustments.

Lapse rate — based on policy duration: higher in early years (8% in the first year), declining over time (2% after 10 years). Adjusted for premium mode (annual payers lapse less than monthly payers).

Disability rate — increases with age above 30, capped at 5%. Female rates are 1.1× male rates.

Step 2: Apply Competing-Risk Decrements

The three hazard rates compete: a policy can only exit through one cause at a time. The total decrement is capped at 1.0 to ensure probabilities remain valid.

Step 3: Compute Expected Cash Flow

The expected cash flow at each time step is:

```
expectedCashflow = probActive × premiumInflow − probDeath × deathBenefit
```

This is a probability-weighted average: the premium inflow is weighted by the probability the policy is still active, and the death benefit outflow is weighted by the probability of death in that period.

Step 4: Update State Probabilities

Three running probabilities are maintained and updated at each step:

- **probActive** — probability the policy is still in force (decreases over time)
- **probDeathClaimed** — cumulative probability of death claim (increases)
- **probLapsed** — cumulative probability of lapse (increases)

These always satisfy: probActive + probDeathClaimed + probLapsed ≤ 1.0

Output Structure

Each time step produces four values per policy (the "4-channel" output):

Channel	Description
ExpectedCashflow	Net probability-weighted cash flow
ProbActive	Probability the policy is still active
ProbDeathClaimed	Cumulative probability of death claim
ProbLapsed	Cumulative probability of lapse

For a portfolio of 10,000 policies projected over 30 years (360 monthly steps), the output is 10,000 × 360 × 4 = 14.4 million values. The GPU produces this in under 50 milliseconds.

GPU Execution

The life insurance kernel maps one GPU thread per policy. Each thread independently projects its policy through all time steps, reading from shared lookup tables and writing to its own output region. Since policies are completely independent, there is no synchronisation between threads — the same parallelism principle that works for banking contracts works equally well for insurance.

For Monte Carlo (stress testing different actuarial assumptions), policies are replicated C×S times before kernel dispatch. Each replica carries scenario-specific adjustments (e.g., a 10% increase in mortality rates). The kernel processes each replica as an independent thread.

Syntax Reference

DSL Syntax Reference

Type System

The DSL is dynamically typed with three value types:

Type	DSL Literals	Example
`f64` (floating-point)	Decimal or integer numbers	`1.35`, `365`, `0.0`
`bool`	Keywords	`true`, `false`
`string`	Double-quoted text	`"Active"`, `"Monthly"`

Integer literals (`24`, `10000`) are automatically promoted to `f64` during evaluation. A non-zero `f64` is truthy in a boolean context.

Lexical Elements

Comments

Single-line comments begin with `//` and run to the end of the line. They are ignored by the lexer.

```
// This is a comment — ignored at runtime
snapshot.age_at_eval  // inline comment
```

Literals

```
1.35          // float literal (f64)
24            // integer literal (promoted to f64)
true          // boolean literal
false         // boolean literal
```

```
"Active"      // string literal
"Monthly"     // string literal
```

Identifiers

Plain identifiers name fields and functions (`snapshot`, `age_at_eval`, `months_between`). Dollar-prefixed identifiers reference previously computed factors (`$factor_age`, `$factor_smoker_loading`).

```
snapshot.age_at_eval            // snapshot field reference
facts.death_reported            // facts field reference
meta.occupation_class           // product-metadata field reference
$factor_base_mortality_qx       // factor reference
$factor_base_mortality_qx@1.0.0 // factor reference pinned to a specific
version
eval_date                       // built-in evaluation-date reference
```

Supported tokens

The lexer recognises the following token kinds:

- Numeric literals — `Float` (`1.35`, `3.65e2`) and `Int` (`24`, `10000`)
- Boolean literals — `BoolTrue` and `BoolFalse`
- String literals — `String` (double-quoted)
- `Identifier` — plain names (`snapshot`, `eval_date`, `months_between`)
- `DollarIdent` — factor references (`$factor_age`)
- Keywords — `if`, `then`, `else`, `rule`, `description`, `inputs`, `guard`, `reason_template`, `priority`
- Arithmetic operators — `+`, `-`, `*`, `/`, `**`
- Comparison operators — `<`, `<=`, `>`, `>=`, `==`, `!=`
- Logical operators — `&&`, `||`, `!`
- Punctuation — `(`, `)`, `[`, `]`, `{`, `}`, `,`, `.`, `:`, `@`

Operators

Arithmetic

Operator	Description	Example
`+`	Addition	`snapshot.premium_amount + 5.0`
`-`	Subtraction	`snapshot.sum_assured - 1000.0`
`*`	Multiplication	`$factor_base_mortality_qx * $factor_smoker_loading`
`/`	Division	`snapshot.extra_prem_bps / 10000.0`
`**`	Exponentiation (right-associative)	`2.0 ** 10.0`
`-` (unary)	Negation	`-snapshot.extra_prem_bps`

Comparison

All comparison operators return `bool`.

Operator	Description	Example
`==`	Equality (numeric or string)	`snapshot.current_state == "Active"`
`!=`	Inequality	`snapshot.insured_gender != "X"`
`<`	Less than	`snapshot.age_at_eval < 18.0`
`<=`	Less than or equal	`snapshot.years_in_force <= 3.0`
`>`	Greater than	`eval_date > snapshot.last_prem_due_days`
`>=`	Greater than or equal	`snapshot.years_in_force >= 3.0`

Logical

Operator	Description	Notes
`&&`	Logical AND (short-circuit)	Right side is skipped if left is `false`
`||`	Logical OR (short-circuit)	Right side is skipped if left is `true`

Operator	Description	Notes
`!`	Logical NOT	`!facts.death_reported`

Short-circuit evaluation is particularly useful in guards where the right-hand side may reference a `facts.*` field that is not always present — the field is never read if the left-hand condition already determines the result.

Operator Precedence

Operators are evaluated in the following order (lowest to highest):

Level	Operator(s)	Associativity
1 (lowest)	`if … then … else`	—
2	`||`	Left
3	`&&`	Left
4	`!`	Prefix
5	`<`, `<=`, `>`, `>=`, `==`, `!=`	Left
6	`+`, `-`	Left
7	`*`, `/`	Left
8	`**`	**Right**
9 (highest)	`-` (unary), literals, references, `(…)`	—

Use parentheses to override precedence explicitly:

```
// Without parens: parsed as (a && b) || c
true && false || true

// Explicit grouping
(snapshot.current_state == "Active"
  || snapshot.current_state == "PaidUp"
  || snapshot.current_state == "ClaimOpen")
&& facts.surrender_requested == true
```

Conditional Expressions

The `if … then … else` construct evaluates one of two branches based on a boolean condition:

```
if <condition> then <value-if-true> else <value-if-false>
```

Only the matching branch is evaluated (short-circuit). Examples:

```
// Smoker loading: 1.35× for smokers, 1.0 for non-smokers
if snapshot.smoker_status then 1.35 else 1.0

// Clamp a value to zero when negative
if snapshot.extra_prem_bps > 0.0 then snapshot.extra_prem_bps else 0.0
```

`if … then … else` has the lowest precedence of all operators, so the condition and both branches are parsed fully before the conditional is resolved. Nesting is allowed but parentheses are recommended for clarity:

```
if snapshot.smoker_status
  then (if snapshot.insured_gender == "M" then 1.50 else 1.35)
  else 1.0
```

Markov State Transitions

Markov State Transitions

Why a Markov Model?

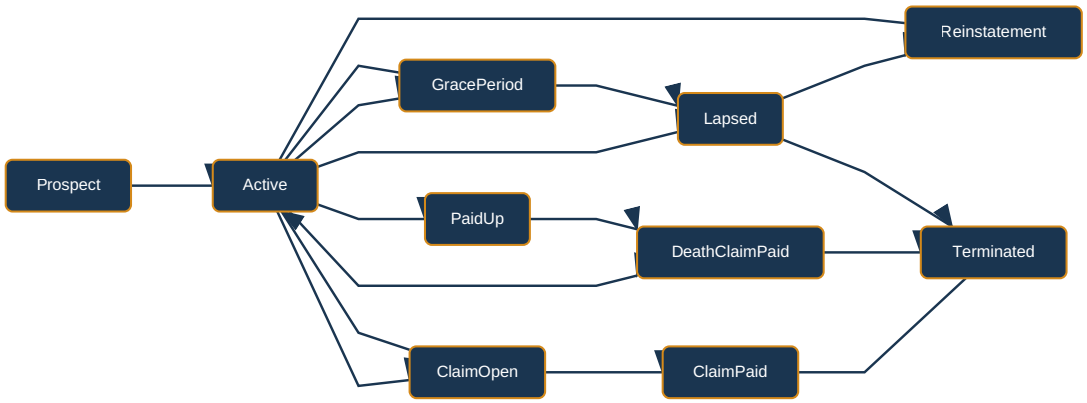
A banking contract follows a fixed schedule. You know in advance exactly when each payment falls and exactly what the cash flow will be. An insurance policy does not work that way. The same policy — same age, same sum assured, same premium — can end in completely different ways: the insured person might die in year three, or lapse the policy in year one, or hold it for thirty years and collect at maturity. There is no single path.

The question the model needs to answer is: given everything we know today, what is the expected cash flow this policy will produce over its lifetime? To answer that you need two things — a complete map of where the policy can go, and a way of assigning probabilities to each possible route.

The Markov model provides exactly that. It defines a set of **states** (the distinct situations a policy can be in) and a set of **transitions** (the moves between states, each with an associated probability). At every point in time, instead of tracking one fixed outcome, the model carries forward a probability for each state simultaneously. A policy does not "choose" to stay active — the model says it is 92% likely to be active, 3% likely to have lapsed, 4% likely to have produced a death claim, and 1% terminated.

The "Markov" property is the key simplification: the probability of moving to the next state depends only on the **current state**, not on how the policy got there. This is both mathematically tractable and actuarially defensible — what matters for a lapse rate is how long the policy has been active, not what happened in an earlier grace period.

The 10 Policy States



State	Code	Terminal	Description
Prospect	0	No	Policy application submitted, not yet active
Active	1	No	Policy is in force, premiums being paid
Grace Period	2	No	Premium missed, within grace period
Lapsed	3	No	Coverage suspended due to non-payment
Paid Up	4	No	Premiums stopped, reduced coverage continues
Claim Open	5	No	A claim has been filed and is being processed
Claim Paid	6	Yes	Claim has been approved and paid
Death Claim Paid	7	Yes	Death benefit has been paid
Reinstatement	8	No	Lapsed policy undergoing reinstatement
Terminated	9	Yes	Policy has ended (final state)

Terminal states absorb the policy — once entered, no further transitions occur.

How the Model Steps Forward

The model runs in monthly time steps across the projection horizon (typically 30 years, so 360 steps). At each step, for each policy, it asks: what is the probability of moving from the current state to each possible next state?

The mechanics follow a straightforward sequence.

Step 1 — Read the current probability distribution. At step zero, the policy is in a known state (say, Active), so the probability for Active is 1.0 and all others are 0.0. As the model runs forward, this spreads into a probability distribution across states: perhaps 85% Active, 8% Lapsed, 4% Death Claim Paid, 3% Terminated.

Step 2 — Look up the hazard rates. For each transition out of the active states, the model retrieves a probability from an actuarial table. These are the primary hazard types:

Hazard Type	Governs	Source
Mortality	Active to Death Claim Paid, Paid Up to Death Claim Paid	Mortality table indexed by age, gender, smoker status
Lapse	Active to Lapsed, Grace Period to Lapsed	Lapse table indexed by duration and premium mode
Disability	Active to Claim Open	Disability table indexed by age and gender

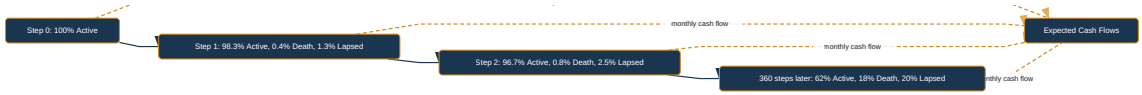
Step 3 — Handle competing risks. Multiple things can happen to the same policy at the same time step, but a policy can only transition once. If the mortality rate is 0.4% and the lapse rate is 5% and the disability rate is 0.8%, the total probability of leaving the Active state is 6.2%. Each destination receives its share of that probability, and the remaining 93.8% stays Active. The model always maintains a valid probability distribution — the probabilities across all states always sum to 1.0.

Step 4 — Compute the expected cash flow. The cash flow at each step is the probability-weighted average:

`expected cash flow = probability(Active) x premium inflow - probability(Death) x death benefit`

This is the key output: not what will happen to this specific policy, but what a large cohort of identical policies would produce on average.

Step 5 — Carry forward and repeat. The updated probability distribution becomes the input for the next time step. Over 360 monthly steps, the distribution gradually migrates — Active probability declines, terminal state probabilities accumulate — producing a complete cash flow timeline from today to the end of the projection horizon.



Transition Hazards

Mortality Model (Gompertz-Makeham)

The mortality hazard follows the Gompertz-Makeham formula, a well-established actuarial model where the force of mortality increases exponentially with age:

```
mu(x) = A + B x exp(x x ln C)
```

Where x is age, and A, B, C are parameters fitted to population data. The annual probability of death is derived as: `q(x) = 1 - exp(-mu(x))`

Adjustments:

- Female mortality: 85% of male mortality
- Smoker loading: 1.35x for smokers
- Underwriting loading: 1 + (ExtraPremBps / 10,000)

Lapse Model

Lapse rates depend on policy duration (years in force) and premium payment mode:

Duration	Base Annual Lapse Rate
0-0.5 years	8%
0.5-1 year	7%
1-2 years	6%
2-3 years	5%
3-5 years	4%
5-10 years	3%
10+ years	2%

Premium mode adjustments: monthly payers have the base rate (1.00x), quarterly payers 0.95x, annual payers 0.90x.

Disability Model

Disability incidence increases with age: base rate of 0.3% + 0.02% for each year over 30, capped at 5%. Female rates are 1.1x male rates.

When Lookup Tables Are Not Enough: Where Rules Come In

The three hazard tables above cover the high-volume transitions well: almost every policy has a mortality rate, a lapse rate, and a disability rate, and they all follow the same parametric or tabular form indexed by a small number of dimensions.

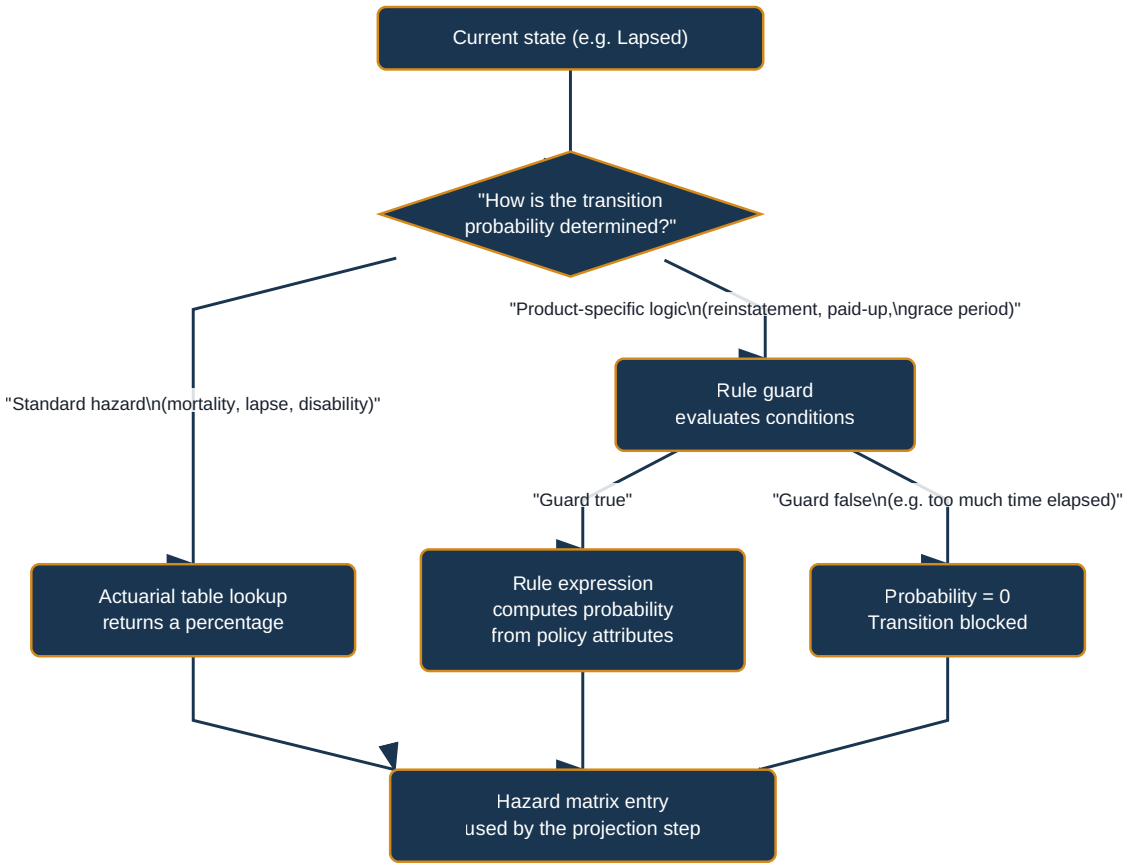
But insurance products are not uniform. Some transitions depend on product-specific logic that a lookup table cannot capture on its own. Consider:

- The grace period length before lapse is triggered: 30 days on some products, 60 on others
- Whether a paid-up conversion is available, and what the reduced sum assured formula is
- Whether reinstatement is permitted after a certain elapsed time since lapse, and what underwriting requirements apply
- A loading that depends on a combination of claims history and years in force

None of these fit a simple actuarial table keyed on age and duration. They depend on the product definition itself.

This is where the **DSL rule set** steps in. Rather than reading a probability directly from a table, the model evaluates a rule. The rule has access to the policy's current attributes and state, can reference actuarial factors, and can read from specialised tables. It returns either a transition probability or a guard condition.

A **guard condition** acts as a gate. It evaluates whether a transition is even possible at this time step. If the guard is false — for example, a reinstatement request arriving more than 24 months after lapse — the transition probability is set to zero regardless of any other calculation. If the guard is true, a further rule expression computes what the probability actually is.



The result is a **hybrid model**: standard actuarial tables handle the dominant probabilistic transitions, and DSL rules handle the product-specific conditional logic. The Markov projection engine itself is indifferent to the source — it receives a completed hazard matrix for each time step and advances the probability distribution. Whether a matrix cell came from a parametric formula, a lookup table, or a DSL expression makes no difference to the step-forward calculation.

This separation means actuaries can introduce new products by writing new rule packs — adjusting grace period lengths, defining conversion options, setting reinstatement windows — without touching the projection engine. The engine is common infrastructure; the product is expressed in rules.

The Markov Graph Definition

The state diagram is defined in an embedded resource (markov_graph.json) that serves as the single source of truth for which states and transitions exist. This file defines:

- **States** — ID, label, and whether the state is terminal

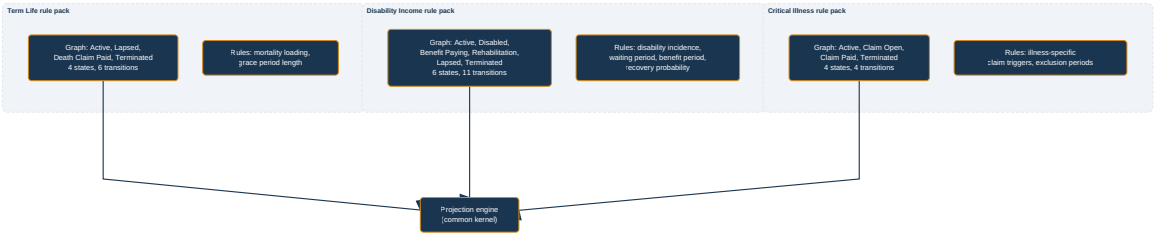
- **Transitions** — from state, to state, hazard type, guard conditions, and effects
- **Effects** — what happens financially when a transition occurs (e.g., "pay death benefit")

By changing the graph definition, different insurance product structures can be modelled — a term life product has different transitions available than a whole life product — without modifying the projection engine. The graph and the rule pack together define the product; the engine is common infrastructure.

One Graph Per Product Type

Each product type carries its own Markov graph. The graph is bundled inside the rule pack alongside the DSL rules and actuarial table references, so when a policy is processed, the engine loads the rule pack for *that* product and uses *that* product's graph. A term life policy and a disability income policy can have entirely different state structures running simultaneously in the same portfolio projection.

This matters because different products genuinely need different topologies. A term life policy is relatively simple — Active, Lapsed, Death Claim Paid, Terminated are the meaningful states. A disability income policy needs additional states such as Disabled, Benefit Paying, and Rehabilitation, plus recovery transitions back to Active that term life does not have at all. A single-premium critical illness contract may not have a Lapsed state at all, since there are no ongoing premiums to miss. Forcing all of these into one universal graph would mean carrying unreachable states and impossible transitions for every product — unnecessary complexity with no benefit.



The projection engine itself is indifferent to which graph it receives. It reads the hazard matrix, steps the probability distribution forward, and writes the cash flows. The graph tells it the shape of the matrix; the rule pack tells it the values. Neither the shape nor the values are baked into the engine.

Mixed portfolios work naturally. When a portfolio contains policies of different product types, each policy is projected using its own rule pack. A batch of 10,000 policies — some term life, some disability income, some critical illness — runs in a single pass. Each policy

carries a reference to its product type; the engine resolves the correct rule pack and graph for each one before the projection starts.

The kernel constraint. The kernel's data structures have a fixed maximum state count, since GPU memory layouts are static. In practice this is resolved by padding: smaller graphs are padded to the maximum size with states that have zero transition probabilities everywhere and are never entered. The projection is mathematically correct regardless — zero-probability states contribute nothing to the probability distribution or the cash flows. The maximum state count is a deployment parameter, not a fundamental limit, and can be increased when a new product requires more states than the current ceiling allows.

Competing Risks

When multiple transitions are possible from the same state (e.g., from Active, a policy could die, lapse, or become disabled), the model uses **competing-risk decrements**. The total transition probability out of a state is the sum of all individual hazard rates, capped at 1.0 to maintain valid probabilities.

This ensures that at every step: probActive + probDeath + probLapse + probDisability = 1.0

Data References

Data References

DSL expressions can read data from five namespaces. Each is described below with the full set of available fields.

`snapshot.*` — Policy snapshot fields

These come directly from the `LifeContractGpu` struct — the current state of the policy at the evaluation date.

DSL field	Type	Description
`snapshot.age_at_eval`	f64	Exact age in years at the evaluation date
`snapshot.sum_assured`	f64	Death / disability benefit amount
`snapshot.premium_amount`	f64	Per-payment premium
`snapshot.years_in_force`	f64	Policy duration in years
`snapshot.extra_prem_bps`	f64	Extra underwriting loading in basis points
`snapshot.lapse_count`	f64	Number of prior lapses
`snapshot.claims_count`	f64	Number of prior claims
`snapshot.lapse_date_days`	f64	Date of last lapse (days since Unix epoch; −1 if never lapsed)
`snapshot.grace_expiry_days`	f64	Grace period expiry date (days since Unix epoch; −1 if not in grace)
`snapshot.last_prem_due_days`	f64	Date the last premium was due (days since Unix epoch)
`snapshot.smoker_status`	bool	`true` for smoker, `false` for non-smoker
`snapshot.insured_gender`	string	`"M"`, `"F"`, or `"X"`
`snapshot.premium_mode`	string	`"Monthly"`, `"Quarterly"`, or `"Annual"`

DSL field	Type	Description
`snapshot.current_state`	string	Current Markov state name (see table below)

Valid `current_state` values

Value	Description
`"Prospect"`	Application received, policy not yet issued
`"Active"`	Policy in force, premiums current
`"GracePeriod"`	Premium overdue; within grace window
`"Lapsed"`	Policy lapsed due to non-payment
`"PaidUp"`	All premiums paid; benefit still in force
`"ClaimOpen"`	Claim filed, pending assessment
`"ClaimPaid"`	Claim settled
`"DeathClaimPaid"`	Death claim settled; policy terminated
`"Reinstatement"`	Lapsed policy undergoing re-underwriting
`"Terminated"`	Policy permanently ended

`facts.*` — Transition context facts

These are event facts supplied at the time of a transition request. They are **not** stored on the policy — they represent what is known at this specific moment in time.

DSL field	Type	Description
`facts.death_reported`	bool	True when a death notification has been received
`facts.claim_filed`	bool	True when a new claim has been filed
`facts.claim_approved`	bool	True when a pending claim has been approved
`facts.uw_approved`	bool	True when underwriting has returned an approval decision

DSL field	Type	Description
`facts.surrender_requested`	bool	True when the policyholder has requested surrender

Because facts are only present when explicitly supplied, guard rules should use `&&` short-circuiting to avoid referencing a facts field unless the left-hand condition is first satisfied:

```
// Safe: right side (facts.death_reported) is only read when state is "Active"
snapshot.current_state == "Active" && facts.death_reported == true
```

`eval_date` — Evaluation date

The current evaluation date as days since the Unix epoch (1970-01-01). Use `months_between` to compute elapsed calendar months.

```
eval_date > snapshot.last_prem_due_days              // has premium due date passed?
months_between(snapshot.lapse_date_days, eval_date) <= 24.0  // within 24 months of lapse?
```

`eval_date` is a scalar value, not a namespace, so it is written without a dot.

`$factor_id` — Computed factor references

A factor expression may reference another factor that has already been evaluated. The engine resolves factor dependencies in topological order so that each factor is guaranteed to be available when it is referenced.

```
$factor_base_mortality_qx * $factor_smoker_loading
```

An optional version suffix pins the reference to a specific rule-pack release, preventing silent behaviour changes when a rule pack is upgraded:

```
$factor_base_mortality_qx@1.0.0
```

Factor references are only valid inside **factor** expressions, not inside guard rules (factors are evaluated before guard evaluation begins, so their final values are available to the engine).

`meta.*` — Product-specific metadata

For products that extend the standard contract model, a `ProductRuleSet<TMeta>` can expose arbitrary typed metadata fields under the `meta.*` namespace. These fields are provided by an `IProductMetaExtractor<TMeta>` implementation at setup time and do not require changes to the core engine.

```
meta.occupation_class          // e.g. "Standard", "HighRisk", "Excluded"
meta.zipcode                   // e.g. "90210"
meta.policy_tier               // e.g. "Gold", "Silver"
```

A typical use case is loading additional risk attributes (such as occupational class or geographic region) that are stored on the product-specific contract extension rather than on the core `LifeContractGpu` struct:

```
// Occupation exclusion guard
!contains("Excluded,HighRisk,Dangerous", meta.occupation_class)

// Region-specific rate adjustment
if meta.policy_tier == "Gold" then 0.95 else 1.0
```

The `meta.*` namespace is empty by default. If a rule references `meta.occupation_class` but no meta extractor is registered, the placeholder `{meta.occupation_class}` is left verbatim in reason templates and the field evaluates to an empty string in expressions.

DSL & Product Rules

DSL & Product Rules

Overview

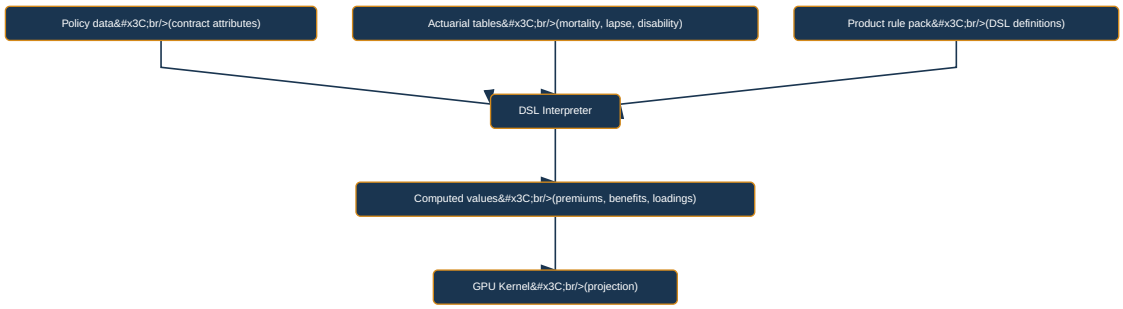
Insurance products vary enormously: a term life policy has different rules from a whole life policy, which has different rules from a critical illness policy. Hard-coding every product's rules into the engine would mean rebuilding the engine every time a new product is launched.

Instead, this project uses a **domain-specific language (DSL)** that allows product rules to be defined in structured text files. The engine interprets these rules at runtime. New products can be added by writing new rule files — no engine changes needed.

What the DSL Expresses

- **Factor expressions** — numeric risk values computed from policy attributes (mortality rates, premiums, benefit amounts)
- **Guard rules** — boolean conditions controlling whether a state transition is allowed
- **Loading adjustments** — modifiers for risk factors such as smoking status or occupational class
- **Table lookups** — references into actuarial tables for mortality, lapse, and disability rates

How Rules Integrate with the Engine



The DSL interpreter runs on the CPU during the setup phase. It evaluates product rules to compute per-policy parameters that are then passed to the GPU kernel as pre-computed scalars. The kernel itself operates on those values — it does not interpret DSL at runtime.

This division keeps the GPU kernel fast and simple (no string processing or dynamic dispatch), while the DSL provides the flexibility to define complex product-specific logic.

Extensibility

Adding a new insurance product requires:

1. Define the product's factors in `factor_set.json` using DSL expressions
2. Define the product's guard rules in `rule_set.json`
3. Provide the associated actuarial lookup tables
4. Define the Markov graph (or reuse an existing one)
5. Package as a rule pack and embed in the assembly

The engine, adapter, and GPU kernel do not need to change.

In This Section

Page	Contents
Syntax Reference	Type system, lexical elements, operators, conditionals
Data References	`snapshot.*`, `facts.*`, `eval_date`, `$factor_id`, `meta.*`
Built-in Functions	`months_between`, `premium_mode_factor`, `contains`, `table_lookup`
Factor Definitions	Factor structure, evaluation order, standard factor set
Guard Rules	Guard structure, reason templates, standard rule set
DSL Internals	Rule packs, compilation pipeline, lexer/parser/interpreter/evaluator

Built-in Functions

Built-in Functions

The DSL provides four built-in functions that cover the most common actuarial operations. All functions return `f64` unless noted otherwise.

`months_between(from_days, to_days)`

Computes the approximate number of calendar months between two dates expressed as days since the Unix epoch (1970-01-01). Uses 30.4375 days per month (= 365.25 ÷ 12).

Returns: `f64`

```
months_between(snapshot.lapse_date_days, eval_date)
// → positive if lapse was in the past; negative if lapse_date_days is in
the future
```

Typical use — check whether a lapsed policy is still within the 24-month reinstatement window:

```
months_between(snapshot.lapse_date_days, eval_date) <= 24.0
```

Note: `snapshot.lapse_date_days` is −1 when the policy has never lapsed, so guards that use this function should first check that `snapshot.lapse_date_days >= 0.0`:

```
snapshot.lapse_date_days >= 0.0
  && months_between(snapshot.lapse_date_days, eval_date) <= 24.0
  && snapshot.lapse_count <= 2.0
```

`premium_mode_factor(mode)`

Returns the number of premium payments per year for a given payment frequency. Accepts either the string name or the numeric code.

Returns: `f64`

Argument	Numeric code	Return value
`"Monthly"`	`0`	`12.0`
`"Quarterly"`	`1`	`4.0`
`"Annual"`	`2`	`1.0`

Typical use — compute the annualised premium from the per-payment amount:

```
snapshot.premium_amount * premium_mode_factor(snapshot.premium_mode)
// → e.g. 150.0 * 12.0 = 1800.0 for a monthly payer
```

`contains(collection, item)`

Tests whether `item` is a member of a comma-separated string list. Returns `bool`.

Returns: `bool`

Comparison is **case-insensitive** and whitespace around commas is trimmed.

```
contains("Excluded,HighRisk,Dangerous", "HighRisk")    // → true
contains("Excluded , HighRisk , Dangerous", "highrisk") // → true
(trimmed + case-insensitive)
contains("Excluded,Dangerous", "Standard")              // → false
```

Typical use — check whether an insured's occupation class falls into an excluded category:

```
// Guard: policy may only be issued if the occupation is not excluded
!contains("Excluded,HighRisk,Dangerous", meta.occupation_class)
```

Or with a soft-loading rule:

```
// Apply a higher loading for occupations in the hazardous list
if contains("Hazardous,HighRisk", meta.occupation_class) then 1.25 else
1.0
```

`table_lookup(table_id, (key1, key2, …))`

Performs an N-dimensional lookup in a named actuarial table. The table must be registered in the rule pack under the given `table_id`. Keys can be any mix of `f64` and `string` values.

Returns: `f64`

```
// 1. Annual mortality probability (qx) — age × gender table
table_lookup("mortality_table_2022", (snapshot.age_at_eval,
snapshot.insured_gender))

// 2. Annual lapse rate — policy duration × payment mode table
table_lookup("lapse_rate_table", (snapshot.years_in_force,
snapshot.premium_mode))

// 3. Disability incidence rate — age × gender table
table_lookup("disability_incidence_table", (snapshot.age_at_eval,
snapshot.insured_gender))
```

How it works

The engine resolves the named table from the rule pack's embedded lookup table registry. For numeric keys the lookup interpolates between adjacent table rows (linear interpolation by default). For string keys the lookup matches an exact row.

Table naming convention

Table identifiers use the pattern `<table_name>_<year>` so that actuarial basis updates are tracked by name:

```
"mortality_table_2022"      // 2022 mortality basis
"mortality_table_2024"      // 2024 update — swap here to change the
basis
"lapse_rate_table"
"disability_incidence_table"
```

A custom `ILookupTableProvider` can be injected into the evaluator to override tables at test time or to substitute environment-specific actuarial bases without changing the DSL expressions.

Monte Carlo Simulation

Monte Carlo Simulation

The Problem with a Single Projection

When you run the Markov model once with your best-estimate actuarial assumptions — your expected mortality table, your expected lapse rates, your expected disability rates — you get one answer: the expected cash flow stream for each policy.

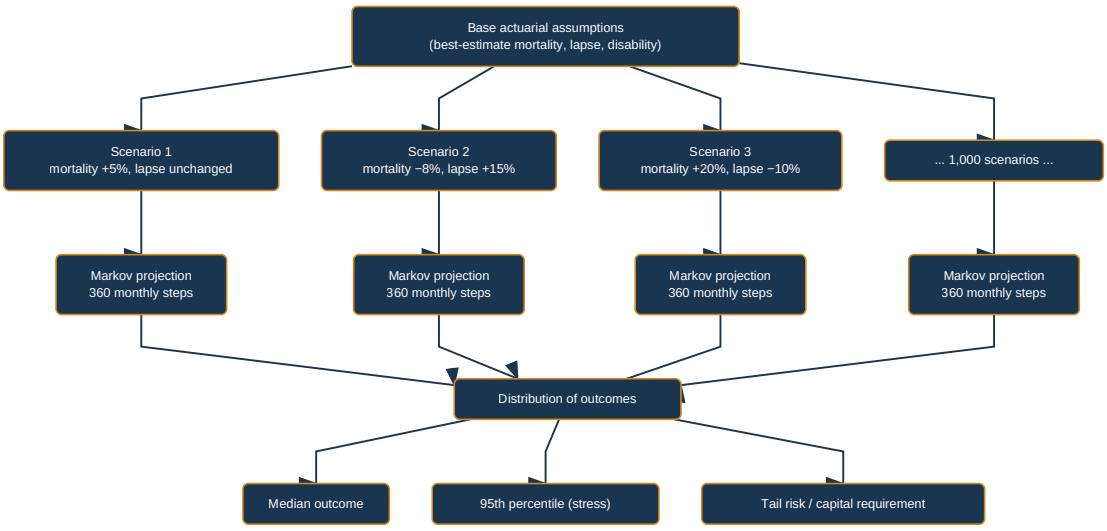
That answer is useful, but it is not enough. Actuarial assumptions are not facts. They are estimates, and they can be wrong. What happens if mortality is 20% higher than expected over the next decade? What if lapse rates spike because interest rates rise and policyholders find better terms elsewhere? What is the realistic downside? How much capital needs to be held against the tail of bad outcomes?

A single projection cannot answer these questions. Monte Carlo simulation can.

What Monte Carlo Means Here

Monte Carlo simulation runs the Markov projection not once but thousands of times. Each run uses a different set of actuarial assumptions — a different "scenario." The collection of results forms a distribution: a picture of the range of possible outcomes and how likely each region of that range is.

The name comes from the idea of repeated random sampling. Rather than trying to enumerate every possible future analytically (which would be intractable), you draw many random scenarios and observe where the results cluster. With enough scenarios, the distribution of outcomes converges to a reliable picture of the risk.



How Scenarios Are Constructed

Each scenario is a set of adjustments applied to the base actuarial assumptions. Common scenario dimensions include:

Mortality shocks — a multiplicative adjustment to the mortality hazard rate across all ages and durations. A scenario might increase mortality by 15% (a pandemic stress), or decrease it by 10% (longevity improvement stress for products that pay on survival).

Lapse shocks — a shift in lapse rates, either flat across all durations or shaped (e.g., higher early lapse under financial stress scenarios where many policyholders can no longer afford premiums).

Disability shocks — an increase or decrease in disability incidence, often correlated with economic cycle scenarios.

Combined shocks — most real-world stress scenarios combine multiple adjustments simultaneously. A recession scenario might combine higher lapses, elevated disability, and slightly elevated mortality together.

The scenarios can be constructed in two ways:

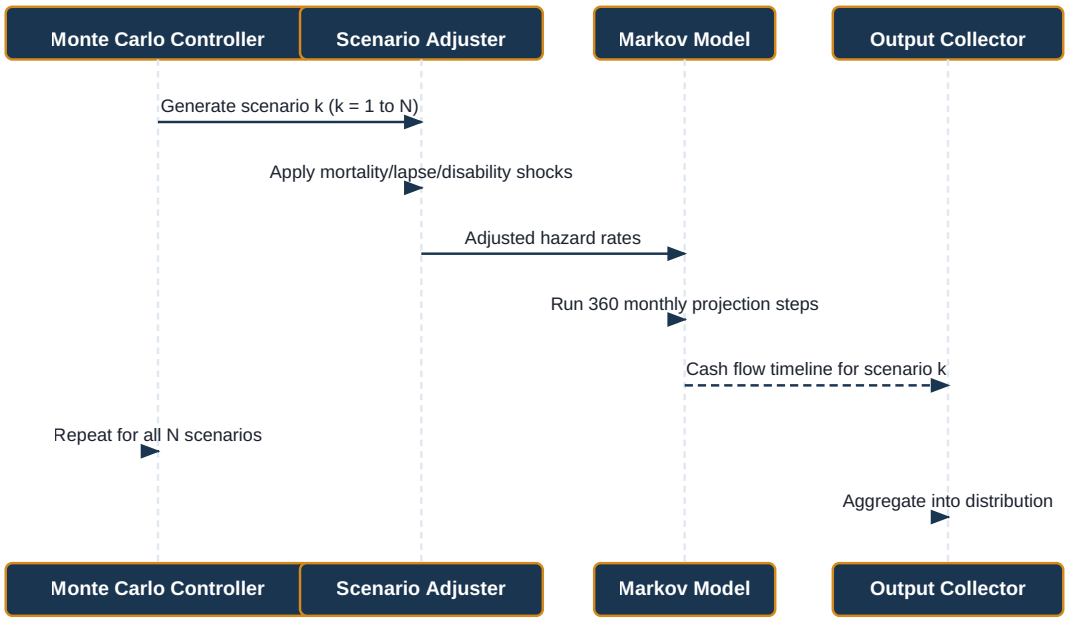
- **Deterministic stress scenarios** — pre-defined shifts that represent specific named risks ("pandemic," "economic downturn," "longevity improvement"). These are used for regulatory stress testing and board-level risk reporting.
- **Stochastic scenarios** — random draws from a probability distribution over the assumption space. Each draw is a plausible but randomly chosen combination of

adjustments. Running thousands of stochastic scenarios produces a full statistical distribution of outcomes.

The Markov Model as the Inner Engine

The critical point is that the Markov model does not change between scenarios. The same 10-state structure, the same transition logic, the same DSL rules, the same competing-risk decrement calculation — all of it runs identically in every scenario. What changes is only the input: the hazard rates fed into the model at each time step.

This is why the architecture separates the projection engine from the actuarial assumptions. The projection engine does one thing: given a set of transition probabilities, step the probability distribution forward. The scenarios are simply different flavours of transition probabilities.



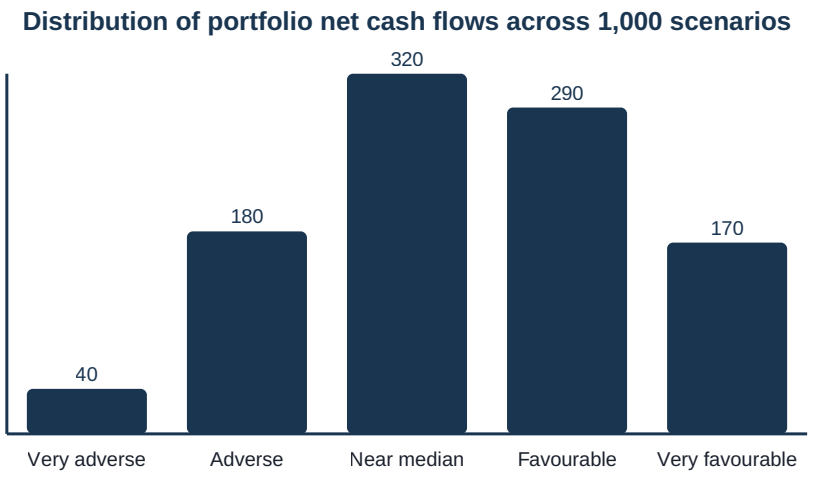
For a portfolio of 10,000 policies, running 1,000 scenarios means executing 10,000 × 1,000 = 10 million independent policy projections. Each projection consists of 360 time steps. This is the computational load that the parallel architecture is designed to absorb: each policy-scenario pair maps to an independent thread, and all threads run simultaneously.

What the Output Looks Like

The result of a Monte Carlo run is a **distribution** over outcomes — typically the total net cash flow for the portfolio across the projection horizon, aggregated across all policies.

For each time step or for the full projection, you can read off:

Statistic	What It Tells You
Median (50th percentile)	The central estimate — what you expect in a typical scenario
75th percentile	A moderately adverse outcome
95th percentile	A severe but plausible adverse scenario — commonly used for capital purposes
99th percentile	A very severe tail event
Mean	The probability-weighted average across all scenarios
Standard deviation	How spread out the outcomes are — a measure of uncertainty



The shape of this distribution matters as much as any single number. A narrow, symmetric distribution indicates stable, predictable outcomes. A wide distribution with a fat left tail indicates significant downside risk. The Monte Carlo output makes this visible.

Stress Testing vs. Full Stochastic

Monte Carlo simulation in an insurance context is used for two related but distinct purposes.

Stress testing runs a specific set of named adverse scenarios and reports the outcomes under each. The goal is to understand the effect of identified risks — what does a 20% increase in mortality do to our solvency position? This is used for regulatory reporting and internal risk management.

Full stochastic projection runs a large number of randomly drawn scenarios and builds a complete probability distribution. The goal is to quantify the full range of uncertainty — not just "what happens if mortality goes up by 20%" but "what is the probability that the portfolio loses more than X?" This is used for economic capital calculations and for pricing the cost of risk.

The same Markov model and the same projection infrastructure serve both purposes. The only difference is how the scenarios are generated and what question is asked of the resulting distribution.

Relationship to the Deterministic Best-Estimate

The single best-estimate Markov projection (running once with base assumptions) is not made redundant by Monte Carlo. It remains the primary number used for reserving and cash flow planning — it represents the most likely outcome under current assumptions.

Monte Carlo adds the uncertainty dimension: it shows how sensitive that number is to assumption changes, and it quantifies the tail risk that the best estimate cannot reveal. The two are complementary: the best-estimate projection tells you where you expect to be; the Monte Carlo distribution tells you how confident you should be in that expectation.

Factor Definitions

Factor Definitions

Factors are numeric intermediate values computed from policy data on the CPU before the GPU kernel runs. They capture everything that varies per policy — mortality rates, premium amounts, benefit sizes — so the GPU kernel receives pre-computed scalars and never needs to interpret DSL.

Factor definition structure

Factors are defined in `factor_set.json`. Each entry looks like this:

```
{
  "id": "factor_smoker_loading",
  "description": "Mortality loading multiplier: 1.35× for smokers, 1.0
for non-smokers",
  "expression": "if snapshot.smoker_status then 1.35 else 1.0",
  "dependencies": [],
  "cache_policy": "per_snapshot",
  "value_type": "f64"
}
```

Field	Description
`id`	Unique factor identifier — referenced in other expressions as `$<id>`
`description`	Human-readable documentation
`expression`	DSL expression that computes the factor value
`dependencies`	Other factor IDs that this expression depends on; drives topological ordering
`cache_policy`	When to recompute: `per_snapshot` (contract changed) or `per_eval_date` (each projection step)
`value_type`	Always `f64` in the current release

Evaluation order

Factors are evaluated in **topological DAG order**: a factor is always evaluated only after all factors listed in its `dependencies` have already been computed. The evaluator uses Kahn's algorithm to determine this order at startup and then evaluates factors in the resulting sequence for every policy.

Each factor's computed value is stored in the `EvalContext.FactorValues` dictionary, where subsequent factors can read it via the `$factor_id` syntax. This means a factor expression like:

```
$factor_base_mortality_qx * $factor_smoker_loading * (1.0 +
snapshot.extra_prem_bps / 10000.0)
```

…will always find `$factor_base_mortality_qx` and `$factor_smoker_loading` already present in the context.

Standard factor set (`life-standard-factors-v1`)

The eight built-in factors for the standard term life product, listed in evaluation order.

`factor_age`

Exact age in years at the evaluation date.

```
snapshot.age_at_eval
```

Dependencies: none

`factor_smoker_loading`

Mortality loading multiplier based on smoking status. Applied to base mortality rates to reflect the higher risk carried by smokers.

```
if snapshot.smoker_status then 1.35 else 1.0
```

Dependencies: none

`factor_base_mortality_qx`

Annual mortality probability (q_x) looked up from a standard 2022 Makeham-Gompertz mortality table, keyed by exact age and gender.

```
table_lookup("mortality_table_2022", (snapshot.age_at_eval,
snapshot.insured_gender))
```

Dependencies: none

`factor_adjusted_mortality_qx`

Base mortality adjusted for smoker status and any extra underwriting loading declared on the policy. `extra_prem_bps` is in basis points (1 bps = 0.01%), so 100 bps = 1% extra loading.

```
$factor_base_mortality_qx
  * $factor_smoker_loading
  * (1.0 + snapshot.extra_prem_bps / 10000.0)
```

Dependencies: `factor_base_mortality_qx`, `factor_smoker_loading`

`factor_lapse_rate`

Annual lapse probability looked up from a duration- and payment-mode-specific table. Policies paid monthly tend to lapse more frequently than annually-paid ones.

```
table_lookup("lapse_rate_table", (snapshot.years_in_force,
snapshot.premium_mode))
```

Dependencies: none

`factor_disability_incidence`

Annual incidence rate for the disability benefit rider, looked up by age and gender and then adjusted for the smoker loading (smokers have a higher disability incidence).

```
table_lookup("disability_incidence_table", (snapshot.age_at_eval,
snapshot.insured_gender))
  * $factor_smoker_loading
```

Dependencies: `factor_smoker_loading`

`factor_benefit_amount`

The sum assured — the face value of the death or disability benefit that will be paid on a valid claim.

```
snapshot.sum_assured
```

Dependencies: none

`factor_annual_premium`

The annualised premium — the per-payment amount scaled by the number of payments per year for the policy's payment frequency.

```
snapshot.premium_amount * premium_mode_factor(snapshot.premium_mode)
```

Examples: a monthly payer paying 150 per month → 1 800 per year; an annual payer paying 1 200 → 1 200 per year.

Dependencies: none

Guard Rules

Guard Rules

Guard rules express boolean conditions that must be satisfied before a Markov state transition is permitted. They gate every event that changes a policy's state — premium lapses, death claims, reinstatements, surrenders, and more.

Guard rule structure

Guard rules are defined in `rule_set.json`. Each entry looks like this:

```
{
  "id": "rule_death_benefit_eligible",
  "description": "Death claim allowed when policy is Active and death is reported",
  "inputs": ["snapshot.current_state", "facts.death_reported"],
  "guard": "snapshot.current_state == \"Active\" && facts.death_reported == true",
  "reason_template": "State {snapshot.current_state}; death reported: {facts.death_reported}.",
  "priority": 100
}
```

Field	Description
`id`	Unique rule identifier, passed to `EvaluateGuard`
`description`	Human-readable documentation
`inputs`	Data fields the guard reads; captured in `GuardResult.InputsUsed` for audit
`guard`	DSL boolean expression; must evaluate to `true` for the transition to proceed
`reason_template`	Message template interpolated at evaluation time (see below)
`priority`	When multiple rules apply to the same transition, the highest priority wins

Reason templates

The `reason_template` string is interpolated using `{…}` placeholders. Any `snapshot.*`, `facts.*`, `meta.*`, or `eval_date` reference is resolved to its runtime value. Unresolved placeholders are left verbatim in the output.

```
"Underwriting approved: {facts.uw_approved}; must be true to proceed."
"Premium due day {snapshot.last_prem_due_days}; eval day {eval_date}."
"Lapse day {snapshot.lapse_date_days}; lapse count
{snapshot.lapse_count}."
"State {snapshot.current_state}; death reported: {facts.death_reported}."
```

The rendered reason is returned as `GuardResult.Reason` and is useful for logging transition decisions and building human-readable audit trails.

`GuardResult`

`EvaluateGuard` returns a `GuardResult` with three fields:

Field	Type	Description
`Allowed`	`bool`	Whether the guard passed
`Reason`	`string`	Rendered reason template
`InputsUsed`	`Dictionary<string, string>`	Values of all declared inputs at evaluation time

`InputsUsed` gives full provenance: you can see exactly what values the engine read when it made its decision.

Standard guard rule set (`life-standard-rules-v1`)

All ten built-in guard rules, in priority order.

`rule_death_benefit_eligible` (priority 100)

Death claim permitted when the policy is in `Active` state and a death notification has been received.

```
snapshot.current_state == "Active" && facts.death_reported == true
```

`rule_claim_approved` (priority 80)

A claim in `ClaimOpen` state transitions to `ClaimPaid` when the claim has been formally approved.

```
snapshot.current_state == "ClaimOpen" && facts.claim_approved == true
```

`rule_claim_admissible` (priority 50)

A general claim is admitted when the policy is `Active` and a claim has been filed.

```
snapshot.current_state == "Active" && facts.claim_filed == true
```

`rule_surrender` (priority 30)

Surrender is allowed from `Active`, `PaidUp`, or `ClaimOpen` states when the policyholder requests it.

```
(snapshot.current_state == "Active"
  || snapshot.current_state == "PaidUp"
  || snapshot.current_state == "ClaimOpen")
&& facts.surrender_requested == true
```

`rule_reinstatement_eligible` (priority 20)

A lapsed policy may apply for reinstatement if the lapse occurred within the past 24 months and the policy has lapsed no more than twice before.

```
snapshot.lapse_date_days >= 0.0
  && months_between(snapshot.lapse_date_days, eval_date) <= 24.0
  && snapshot.lapse_count <= 2.0
```

`rule_reinstatement_approved` (priority 20)

A policy in `Reinstatement` state transitions back to `Active` after successful re-underwriting.

```
snapshot.current_state == "Reinstatement" && facts.uw_approved == true
```

`rule_underwriting_approved` (priority 10)

A policy may only be issued (transition from `Prospect` to `Active`) when underwriting has returned an approval decision.

```
facts.uw_approved == true
```

`rule_paid_up_eligible` (priority 10)

A policy may convert to paid-up status only after the minimum 3-year premium-paying term has been completed.

```
snapshot.current_state == "Active" && snapshot.years_in_force >= 3.0
```

`rule_premium_overdue` (priority 5)

Triggers the transition from `Active` to `GracePeriod` when the last premium due date has passed.

```
eval_date > snapshot.last_prem_due_days && snapshot.current_state ==
"Active"
```

`rule_grace_period_expired` (priority 5)

Triggers a lapse when the grace period window has expired without a premium payment.

```
snapshot.grace_expiry_days > 0.0 && eval_date >
snapshot.grace_expiry_days
```

Adding a custom guard rule

To add a new rule, append an entry to `rule_set.json`:

```
{
  "id": "rule_age_limit",
  "description": "Policy cannot be issued if insured age exceeds 65",
  "inputs": ["snapshot.age_at_eval"],
  "guard": "snapshot.age_at_eval <= 65.0",
  "reason_template": "Insured age {snapshot.age_at_eval} must be <= 65
for new issues.",
  "priority": 15
}
```

No engine changes are needed. The rule pack is reloaded automatically on next startup.

DSL Internals

DSL Internals

This page describes how the DSL is loaded, compiled, and evaluated at runtime — covering rule packs, the compilation pipeline, and the four main components: lexer, parser, interpreter, and rule evaluator.

Rule Packs

Rules and factors are packaged into **rule packs** — a pair of JSON files that together define a complete insurance product:

File	Contents
`rule_set.json`	All transition guard rules for the product
`factor_set.json`	All risk factor definitions for the product

Each pack has a unique `id` (e.g. `life-standard-rules-v1`) and a semver `version`. Multiple products can coexist in the same assembly — each policy's product type determines which pack is used.

Rule packs are loaded from embedded assembly resources at startup by `DslRulePackLoader`. The loader parses every expression and guard with `DslParser` and caches the resulting ASTs. Parsing happens once; all subsequent evaluations operate directly on the cached AST.

Compilation Pipeline

The four stages map directly to four classes:

Stage	Class	Responsibility
Tokenisation	`DslLexer`	Converts a DSL source string into a `Token` stream
Parsing	`DslParser`	Converts the token stream into an AST
Evaluation	`DslInterpreter`	Tree-walks the AST against an `EvalContext`
Orchestration	`DslRuleEvaluator`	Loads rules/factors, manages topological order, exposes public API

Lexer (`DslLexer`)

`DslLexer` tokenises a rule expression string into a flat list of `Token` values. Each token carries a `Kind` (enum) and `Text` (the matched source string). Single-line `//` comments are silently discarded.

Example tokens from `snapshot.age_at_eval`:

Token	Kind
`snapshot`	`Identifier`
`.`	`Dot`
`age_at_eval`	`Identifier`

Example tokens from `$factor_smoker_loading`:

Token	Kind
`factor_smoker_loading`	`DollarIdent`

Parser (`DslParser`)

`DslParser` implements a **recursive-descent grammar** with explicit operator precedence. It produces an Abstract Syntax Tree (AST) whose node types map one-to-one with DSL constructs:

AST Node	Example source
`FloatLiteralExpr`	`1.35`
`IntLiteralExpr`	`24`
`BoolLiteralExpr`	`true`, `false`
`StringLiteralExpr`	`"Active"`
`SnapshotFieldExpr`	`snapshot.age_at_eval`
`EvalDateExpr`	`eval_date`
`FactsFieldExpr`	`facts.death_reported`
`MetaFieldExpr`	`meta.occupation_class`
`FactorRefExpr`	`$factor_smoker_loading`
`BinaryOpExpr`	`a + b`, `x >= 3.0`, `p && q`
`UnaryOpExpr`	`-x`, `!flag`
`IfExpr`	`if cond then a else b`
`TableLookupExpr`	`table_lookup("tbl", (k1, k2))`
`BuiltinCallExpr`	`months_between(a, b)`, `premium_mode_factor(m)`

The parser entry point is `DslParser.ParseExpression()`. It never mutates shared state and is safe to call from multiple threads.

Interpreter (`DslInterpreter`)

`DslInterpreter` is a **tree-walking interpreter** that evaluates an AST node against an `EvalContext`. The static method `DslInterpreter.Eval(expr, ctx)` is the single entry point.

Key evaluation rules:

- Integer literals are promoted to `DslDouble` (f64) automatically
- `&&` and `||` are **short-circuit**: the right branch is never evaluated if the left branch determines the result
- `==` performs cross-type equality: comparing a `DslBool` to a `DslDouble` is valid

- Any unresolvable field access (e.g. a `facts.*` field that was not supplied) throws `DslRuntimeException`
- Type mismatches in arithmetic (e.g. `"Active" + 1.0`) throw `DslRuntimeException`

The interpreter is **stateless** — all mutable state lives in the `EvalContext` that is passed in. It is therefore thread-safe: multiple threads can evaluate the same AST simultaneously, each with their own context.

Rule Evaluator (`DslRuleEvaluator`)

`DslRuleEvaluator` is the high-level entry point for the rest of the engine. It is constructed once (a thread-safe singleton is available as `DslRuleEvaluator.Default`) and exposes two public methods.

`EvaluateGuard(ruleId, ctx)`

Evaluates the named guard rule and returns a `GuardResult`:

```
var result =
DslRuleEvaluator.Default.EvaluateGuard("rule_death_benefit_eligible",
ctx);
// result.Allowed    → true / false
// result.Reason     → "State Active; death reported: True."
// result.InputsUsed → { "snapshot.current_state": "Active",
"facts.death_reported": "True" }
```

Throws `InvalidOperationException` if the rule ID is not found or if the guard expression evaluates to a non-boolean type.

`EvaluateFactors(ctx)`

Evaluates all factors in topological DAG order, populating `ctx.FactorValues` incrementally:

```
DslRuleEvaluator.Default.EvaluateFactors(ctx);
// ctx.FactorValues["factor_age"]                  → 54.0
// ctx.FactorValues["factor_smoker_loading"]        → 1.35
// ctx.FactorValues["factor_adjusted_mortality_qx"]  → <computed>
```

Each factor stores its result into `ctx.FactorValues` as it is computed, making it immediately available to any factor that depends on it later in the sequence. The method returns a snapshot copy of the dictionary.

Topological sort

At construction time the evaluator runs Kahn's algorithm over the factor dependency graph to determine evaluation order. This is a one-time O(V+E) cost. If a cycle exists in the dependencies, construction fails with an `InvalidOperationException`.

Evaluation Context (`EvalContext`)

`EvalContext` carries everything a DSL expression can read:

```
new EvalContext
{
    SnapshotFields = /* IReadOnlyDictionary<string, DslValue> — from
LifeContractGpu */,
    FactsFields    = /* IReadOnlyDictionary<string, DslValue> — from
TransitionContextFacts */,
    MetaFields     = /* IReadOnlyDictionary<string, DslValue> — from
IProductMetaExtractor */,
    EvalDateDays   = /* double — days since Unix epoch */,
    FactorValues   = /* Dictionary<string, double> — populated during
EvaluateFactors */,
    Tables         = /* ILookupTableProvider — for table_lookup
expressions */,
}
```

Use `EvalContextBuilder.Build(contract, evalDateDays, facts)` to construct a standard context, or `EvalContextBuilder.BuildWithMeta(...)` when product-specific metadata fields are needed.